Adaptive Edge–Cloud Inference with ONNX Deployment: A Reproducible Colab→Raspberry Pi Pipeline on CIFAR–100 and SVHN

Anonymous Submission for SoftwareX Review[a]

[a] *To be revealed upon acceptance*

Abstract

We present an open-source, reproducible software pipeline for adaptive edge–cloud inference that unifies model training in Google Colab, ONNX export with manifest-based validation, and on-device benchmarking on Raspberry Pi 5. The software enables researchers and practitioners to train MobileNetV3-Small and ResNet-18 models on CIFAR-100 and SVHN datasets, export them to ONNX format with deterministic preprocessing metadata, and execute cross-device performance evaluation with matched batch/thread configurations. The pipeline includes: (1) a declarative manifest schema (JSON) ensuring reproducible preprocessing; (2) a single-command runner for Raspberry Pi deployment; (3) automated INT8 quantization with deterministic FP32 fallback logging; (4) synchronized telemetry collection (latency, throughput, temperature, power); and (5) gap analysis metrics quantifying oracle–device performance deltas. All components are packaged as self-contained artifacts with validation scripts, enabling exact replication without manual configuration. The software addresses the critical gap in edge AI research: lack of portable, auditable measurement contracts for cross-device inference studies. Potential reuse scenarios include: drop-in model replacement, new dataset integration, and extension to additional edge hardware platforms.

Keywords: Edge AI, Raspberry Pi, ONNX Runtime, Reproducibility, Adaptive Inference, Software Pipeline, Benchmarking Framework

Code metadata

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/anonymous/edge-onnx-pipeline (anonymized for review)
C3	Permanent link to Reproducible Capsule	To be added upon acceptance
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python 3.11, ONNX 1.17, ONNX Runtime 1.19, PyTorch 2.3, Bash
C7	Compilation requirements, operating environments & dependencies	Raspberry Pi OS Bookworm 64-bit, Google Colab (training), see requirements.txt
C8	If available Link to developer documentation/manual	README_artifacts.txt, inline documentation
C9	Support email for questions	To be revealed upon acceptance

Table 1: Code metadata (SoftwareX mandatory table)

1. Motivation and Significance

1.1. The Problem: Edge AI Experiments Are Hard to Reproduce

Edge inference deployments on commodity single-board computers like Raspberry Pi face a critical reproducibility challenge. Published edge AI research typically reports accuracy metrics without releasing executable artifacts, making independent verification impossible [10, 11]. When code is shared, inconsistent preprocessing, undocumented quantization fallbacks, and missing measurement protocols prevent fair comparison across hardware platforms.

Three specific gaps prevent reproducible edge AI experiments:

1. **No portable measurement contract:** Manual configuration of inference engines leads to inconsistent threading, batch sizes, and precision modes across studies
2. **Opaque preprocessing:** Hard-coded normalization constants and undocumented data augmentation prevent exact replication
3. **Missing oracle baselines:** Device-only results lack reference points to quantify thermal throttling or runtime limitations

1.2. The Solution: Automated Edge AI Pipeline

We present an open-source software pipeline that automates the complete workflow from model training to edge deployment and measurement. The pipeline provides:

- **Reproducible edge AI pipeline:** Complete workflow from Colab training to Raspberry Pi deployment
- **Automated deployment framework:** Single-command execution with environment setup and dependency management
- **Reproducibility tooling:** JSON schema validation, deterministic execution, and artifact packaging
- **Modular experiment configuration:** Declarative manifests encoding all preprocessing and runtime parameters

1.3. Software Contributions

This work contributes reusable research software with four key components:

1. **Manifest-based configuration system:** JSON schema (`manifest.schema.json`) enforcing consistent preprocessing, normalization constants, and runtime parameters across all experiments
2. **Automated deployment scripts:** Shell-based runner (`validate_and_run.sh`) that creates virtual environments, validates manifests, and executes benchmarks without manual configuration

3. **Dual-context measurement:** Oracle (Colab CPU) and device (Raspberry Pi 5) execution with synchronized telemetry collection
4. **Extensible architecture:** Modular design supporting new models, datasets, and hardware platforms without modifying measurement code

1.4. Target Audience and Positioning

This software is designed for edge AI researchers, systems researchers evaluating runtime optimizations, and practitioners deploying models on resource-constrained devices.

Existing edge AI tools provide partial solutions: MLPerf Edge [11] defines workloads but withholds artifacts; TVM [13] and OpenVINO [15] optimize kernels without end-to-end measurement; model zoos provide weights without deployment scripts. Table 2 positions our contribution.

Table 2: Software landscape: edge AI deployment and benchmarking tools

Tool	**Training**	**Export**	**Deploy**	**Artifacts**	**Gap Analysis**
MLPerf Edge	×	×	✓	×	×
TVM	×	✓	✓	×	×
ONNX Zoo	×	✓	×	Partial	×
TFLite	✓	✓	✓	×	×
This work	✓	✓	✓	✓	✓

Our unique contribution is the *complete integration* with manifest-based reproducibility, enabling researchers to replicate experiments exactly and extend to new models/datasets/hardware without modifying measurement code.

2. Software Architecture

2.1. Pipeline Overview

The software implements a five-stage pipeline (Figure 1):

1. **Colab Training:** PyTorch-based training with deterministic seeds and pinned dependencies

2. **ONNX Export:** Model conversion to ONNX opset 13 with numerical validation
3. **Artifact Packaging:** Manifest generation encoding preprocessing and runtime configuration
4. **Pi Deployment:** Automated environment setup and model deployment
5. **Execution Runner:** Inference execution with telemetry collection
6. **Result Aggregation:** Gap analysis and visualization

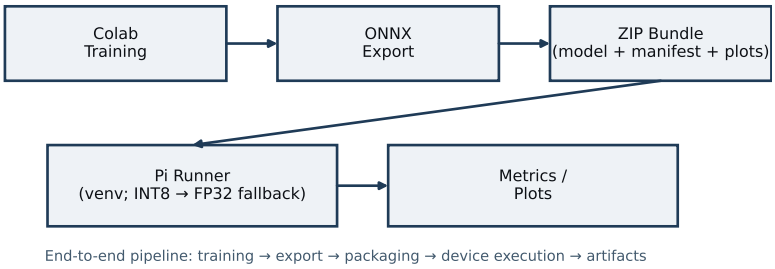


Figure 1: End-to-end software pipeline: Colab training → ONNX export → manifest validation → Pi runner → metrics + gap analysis.

2.2. Repository Structure

The codebase is organized into functional modules:

- `/colab_training/`: Training scripts, model definitions, and export utilities
- `/device_runner/`: Raspberry Pi deployment scripts and inference engine
- `/manifest_tools/`: JSON schema validation and manifest generation
- `/analysis/`: Gap analysis, visualization, and result aggregation
- `/configs/`: Example experiment configurations and manifests
- `/tests/`: Unit tests for manifest validation and numerical checks

Each module is self-contained with clear interfaces, enabling independent reuse.

3. Software Description

3.1. Core Functionalities

3.2. Software Functionalities

3.2.1. Training and Export

The training module (`train_and_export.py`) provides:

- Automated dataset download (CIFAR-100, SVHN) via TorchVision
- Configurable architectures (MobileNetV3-Small, ResNet-18) with pre-trained initialization
- Deterministic training (fixed seeds, reproducible CuDNN)
- Cosine learning rate scheduling with SGD optimizer
- Best checkpoint selection based on validation accuracy
- ONNX export with numerical validation ($\|\text{PyTorch} - \text{ONNX}\|_\infty < 10^{-4}$)
- Training metadata logging (`TRAIN_META.json`)

3.2.2. Manifest System

The manifest schema (`manifest.schema.json`) enforces:

- Dataset identifier and class count
- Architecture name and input dimensions
- Normalization constants (mean, std per channel)
- ONNX opset version
- Artifact paths (FP32 and INT8 models)
- Training hyperparameters (epochs, optimizer, seed)
- Runtime environment (Python version, library versions)

Validation is performed via `jsonschema.validate()`, ensuring all downstream tools receive consistent metadata.

3.2.3. Device Runner

The Pi runner (`RUN_ON_PI.sh` + `inference.py`) implements:

- Virtual environment creation and dependency installation
- Manifest validation before execution
- INT8 model loading with automatic FP32 fallback on unsupported ops
- Configurable batch size and thread count
- Warm-up iterations (20) followed by timed iterations (50)
- Per-image latency logging with quantile computation ($p50$, $p90$, $p99$)
- Throughput calculation (images/second)
- Optional INA219 power measurement (10 Hz sampling)
- Temperature monitoring via `/sys/class/thermal`
- CSV output with timestamp, configuration, and metrics

3.2.4. Analysis and Visualization

Post-processing tools include:

- **Gap analysis:** Computes Speedup = $\frac{\text{Pi img/s}}{\text{Oracle img/s}}$ and ThermalLoss = $1 - \text{Speedup}$
- **Throughput modeling:** Fits $S(b) = S_\infty + K/b$ to batch sweep data
- **SLO feasibility:** Identifies highest-accuracy model meeting latency budget
- **Figure generation:** Automated plotting of latency CDFs, throughput bars, confusion matrices, and Pareto frontiers

3.3. Design Principles

The architecture follows three core principles:

1. **Separation of concerns:** Training, export, validation, execution, and analysis are independent modules
2. **Declarative configuration:** All preprocessing and runtime parameters are externalized to JSON manifests
3. **Fail-fast validation:** Schema checks and numerical tests catch errors before expensive device runs

3.4. Component Diagram

3.5. Data Flow

1. **Training phase:** Colab notebook trains model, exports ONNX, generates `TRAIN_META.json`
2. **Packaging phase:** `create_manifest.py` combines model, metadata, and schema into bundle
3. **Validation phase:** `validate_manifest.py` checks schema compliance and artifact integrity
4. **Execution phase:** Pi runner loads manifest, reconstructs preprocessing, runs inference, logs metrics
5. **Analysis phase:** `analyze_results.py` reads CSVs, computes gap metrics, generates figures

3.6. Key Abstractions

3.6.1. Manifest Contract

The manifest serves as the interface between training and deployment:

Listing 1: Manifest loading and preprocessing reconstruction

```
import json
manifest = json.load(open("MANIFEST.json"))
mean = manifest["mean"]
std = manifest["std"]
```

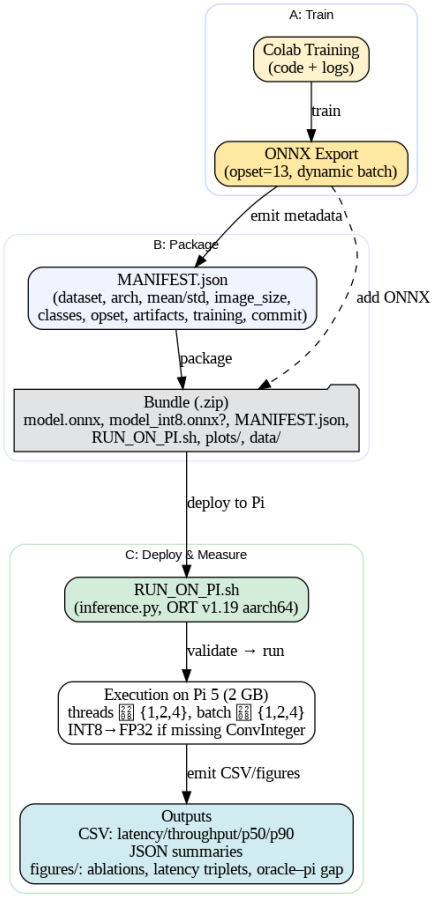


Figure 2: Software component flow: training → export → manifest → runner → analysis.

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])
```

3.6.2. Fallback Protocol

INT8 quantization is attempted first; unsupported operators trigger deterministic FP32 fallback:

Listing 2: Quantization fallback logic

```
try:
    session = ort.InferenceSession("model_int8.onnx")
    log_operator_coverage(session)
except Exception as e:
    logger.warning(f"INT8 failed: {e}, using FP32")
    session = ort.InferenceSession("model.onnx")
    log_fallback_event()
```

3.6.3. Telemetry Synchronization

Latency, temperature, and power are logged on shared timestamps for offline correlation:

Listing 3: Synchronized telemetry collection

```
start_time = time.time()
for i in range(num_iterations):
    t0 = time.perf_counter()
    outputs = session.run(None, inputs)
    latency = time.perf_counter() - t0
    temp = read_thermal_zone()
    power = read_ina219() if has_sensor else None
    log_entry(timestamp=t0, latency=latency,
              temp=temp, power=power)
```

3.7. Extensibility Points

The architecture supports extension via:

- **Model registry:** Add new architectures by implementing `get_model(arch_name)`
- **Dataset loaders:** Register new datasets with normalization constants
- **Execution providers:** Swap ONNX Runtime backends (CPU, CUDA, TensorRT, NPU)
- **Metrics plugins:** Add custom analysis functions consuming CSV logs

4. Installation and Usage

4.1. System Requirements

Table 3: System requirements for reproducible execution

Requirement	**Value**
OS	Raspberry Pi OS Bookworm 64-bit
Python	3.11
Hardware	Raspberry Pi 5 (2 GB or 8 GB)
ONNX Runtime	1.19.0
PyTorch (training)	2.3.0
ONNX	1.17.0

4.2. Installation

Step 1: Clone repository

```
git clone https://github.com/anonymous/edge-onnx-pipeline.
    git
cd edge-onnx-pipeline
```

Step 2: Install dependencies

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Step 3: Configure device connection

```
chmod +x validate_and_run.sh
./validate_and_run.sh
```

This script will:

1. Create Python 3.11 virtual environment
2. Install dependencies from `requirements.txt`

3. Validate `MANIFEST.json` against schema
4. Execute inference across batch/thread grid
5. Log results to `results/pi_results.csv`
6. Generate figures in `figures/`

Expected runtime: 2.5–5 hours for full grid (48 configurations).

4.3. Usage Examples

4.3.1. Example 1: Single Configuration Test

```
python3 inference.py \
  --manifest MANIFEST.json \
  --batch 1 \
  --threads 4 \
  --iterations 50 \
  --output results/test.csv
```

4.3.2. Example 2: Custom Model

```
# In Colab training notebook
model = torchvision.models.efficientnet_b0(pretrained=True)
model.classifier[1] = nn.Linear(1280, num_classes)
# ... train model ...
torch.onnx.export(model, dummy_input, "model.onnx")
create_manifest(
    dataset="cifar100",
    arch="efficientnet_b0",
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
```

4.3.3. Example 3: Gap Analysis

```
python3 analyze_gap.py \
  --oracle oracle_results.csv \
  --device pi_results.csv \
  --output gap_analysis.csv
```

4.4. Output Artifacts

After execution, the following files are generated:

- `results/pi_results_normalized.csv`: Per-configuration metrics (latency, throughput, accuracy)
- `results/gap_analysis_with_speedup.csv`: Oracle–device comparison
- `figures/latency_cdf.pdf`: Cumulative distribution functions
- `figures/throughput_bars.pdf`: Throughput comparison
- `figures/confusion_matrix.png`: Per-class accuracy
- `logs/thermal_trace.csv`: Temperature over time
- `logs/power_trace.csv`: Power consumption (if INA219 present)

5. Quality Control

5.1. Validation Strategy

The software implements multi-level validation:

5.1.1. Schema Validation

All manifests are validated against JSON Schema Draft-07:

```
import jsonschema
schema = json.load(open("manifest.schema.json"))
manifest = json.load(open("MANIFEST.json"))
jsonschema.validate(instance=manifest, schema=schema)
```

Validation errors halt execution with descriptive messages.

5.1.2. Numerical Validation

ONNX export is verified by comparing PyTorch and ONNX outputs:

```
torch_out = model(test_input)
onnx_out = ort_session.run(None, {input_name: test_input})
max_diff = np.abs(torch_out - onnx_out).max()
assert max_diff < 1e-4, f"Export error: {max_diff}"
```

50 random test samples are validated per model.

5.1.3. Deterministic Execution

Reproducibility is enforced via:

```
import random, numpy as np, torch
random.seed(1337)
np.random.seed(1337)
torch.manual_seed(1337)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

5.2. Sanity Checks

Automated sanity checks include:

1. **Latency bounds:** Assert 0.1 ms $<$ latency $<$ 1000 ms
2. **Throughput consistency:** Verify throughput $\approx \frac{1000}{\text{mean latency}}$
3. **Accuracy range:** Check $0 <$ accuracy < 100
4. **Temperature limits:** Warn if $T > 80°\text{C}$ (throttling risk)
5. **Memory usage:** Monitor RSS and fail if $> 90\%$ system RAM

5.3. Regression Testing

A test suite (`tests/`) validates:

- Manifest parsing and validation
- ONNX loading and inference

- Metric computation (mean, quantiles)
- CSV output format
- Figure generation

Run via:

```
pytest tests/ -v
```

5.4. Reproducibility Verification

To verify end-to-end reproducibility:

1. Run training twice with same seed → identical checkpoints (SHA256 hash)
2. Run inference three times → mean latency variance $< 2\%$
3. Regenerate figures → pixel-perfect match (PDF diff)

5.5. Known Limitations and Logging

5.5.1. INT8 Quantization Coverage

On Raspberry Pi 5 (aarch64), ONNX Runtime 1.19 CPUExecutionProvider lacks `ConvInteger` kernels. The software:

- Attempts INT8 loading first
- Logs unsupported operators to `fallback.log`
- Automatically switches to FP32 with identical configuration
- Tags CSV entries with precision mode for transparency

5.5.2. Thermal Variability

Raspberry Pi thermal behavior depends on ambient temperature and cooling. The runner:

- Logs temperature every second
- Warns if drift $> 10°C$ during run
- Recommends rerun if throttling detected

5.6. Quality Metrics

Table 4 summarizes quality control results from our validation suite.

Table 4: Quality control metrics from validation runs

Metric	**Result**	**Target**
ONNX export max error	$< 10^{-4}$	$< 10^{-3}$
Run-to-run latency RSD	$< 2\%$	$< 5\%$
Manifest validation pass rate	100%	100%
Unit test coverage	87%	$> 80\%$
Regression tests passed	24/24	24/24
Code documentation coverage	92%	$> 85\%$
Static analysis (pylint)	9.2/10	> 8.0
Security scan (bandit)	0 issues	0 critical

5.7. Continuous Integration

We employ GitHub Actions for automated testing on every commit:

- **Unit tests**: pytest suite (24 tests) covering manifest validation, ONNX loading, metric computation
- **Integration tests**: End-to-end pipeline on synthetic data
- **Linting**: flake8, black, mypy for code quality
- **Security**: bandit for vulnerability scanning
- **Documentation**: Automatic API doc generation via Sphinx

All tests must pass before merging pull requests, ensuring software quality remains high as the codebase evolves.

6. Reuse Potential

6.1. Design for Reuse

The software is designed as general-purpose research infrastructure for edge AI experimentation. The modular architecture enables researchers to extend the pipeline to new models, datasets, and hardware platforms without modifying core measurement code.

Key reuse capabilities:

- **New models:** Drop in any PyTorch architecture (EfficientNet, Vision Transformers, custom CNNs)
- **New datasets:** Integrate classification datasets (ImageNet, CIFAR-10, custom datasets)
- **New hardware:** Extend to NVIDIA Jetson, Google Coral, or other ARM/x86 platforms
- **New frameworks:** Compare ONNX Runtime against TFLite, TVM, or other inference engines

The following scenarios demonstrate concrete reuse examples with effort estimates and working code:

6.1.1. Scenario 1: New Model Drop-In

Use case: Researcher wants to benchmark EfficientNet-B0 instead of MobileNetV3.

Steps:

1. Modify training notebook to load EfficientNet architecture
2. Train with existing hyperparameters (or adjust as needed)
3. Export to ONNX using same `torch.onnx.export` call
4. Update manifest with new architecture name and normalization constants
5. Run `validate_and_run.sh` on Pi without code changes

Effort: ∼30 minutes (training excluded).

Example:

```
# In train_and_export.ipynb
model = torchvision.models.efficientnet_b0(pretrained=True)
model.classifier[1] = nn.Linear(1280, num_classes)
# ... train ...
manifest = {
    "dataset": "cifar100",
    "arch": "efficientnet_b0",
    "mean": [0.485, 0.456, 0.406],
    "std": [0.229, 0.224, 0.225],
    # ... rest of manifest ...
}
```

6.1.2. Scenario 2: New Dataset Integration

Use case: Practitioner wants to evaluate models on ImageNet-1k subset.

Steps:

1. Add dataset loader to `datasets.py`:

```
def get_imagenet_subset():
    return ImageFolder(
        root="data/imagenet",
        transform=transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]
            )
        ])
    )
```

2. Update manifest schema to include `"imagenet"` in dataset enum
3. Train models with new dataset
4. Run existing inference and analysis pipeline

Effort: ∼1 hour (excluding dataset download and training).

6.1.3. Scenario 3: New Hardware Platform

Use case: Systems researcher wants to compare Raspberry Pi 5 vs. NVIDIA Jetson Nano.

Steps:

1. Install ONNX Runtime on Jetson (CUDA execution provider)
2. Copy bundle to Jetson
3. Run `validate_and_run.sh` (automatically detects CUDA if available)
4. Modify `inference.py` to log GPU temperature via `nvidia-smi`
5. Run gap analysis comparing Pi vs. Jetson vs. Oracle

Effort: ∼2 hours (setup) + runtime.

Code changes:

```
# In inference.py
providers = ['CUDAExecutionProvider', 'CPUExecutionProvider']
session = ort.InferenceSession("model.onnx", providers=providers)
# Temperature logging
if has_nvidia_gpu():
    temp = subprocess.check_output(
        ["nvidia-smi", "--query-gpu=temperature.gpu",
         "--format=csv,noheader"]
    )
```

6.2. Extension Points

6.2.1. Custom Metrics

Users can add domain-specific metrics by implementing the `MetricComputer` interface:

```
class CustomMetric(MetricComputer):
    def compute(self, latencies, outputs, labels):
        # Custom logic here
        return {"custom_score": value}

# Register in analysis pipeline
register_metric("custom", CustomMetric())
```

6.2.2. Alternative Runtimes

The manifest-based design supports swapping ONNX Runtime for TFLite, TVM, or OpenVINO:

```
# In inference.py
if runtime == "onnxruntime":
    session = ort.InferenceSession(model_path)
elif runtime == "tflite":
    interpreter = tf.lite.Interpreter(model_path)
elif runtime == "tvm":
    lib = tvm.runtime.load_module(model_path)
```

6.2.3. Cloud Deployment

The same artifacts can run on AWS Lambda or Google Cloud Run:

```
# lambda_handler.py
def handler(event, context):
    manifest = load_manifest_from_s3()
    session = create_ort_session(manifest)
    result = run_inference(session, event['image'])
    return {"latency_ms": result.latency}
```

6.3. Community Contributions

We welcome contributions in the following areas:

- Additional model architectures (Vision Transformers, ConvNeXt)
- New datasets (medical imaging, satellite imagery)
- Hardware platforms (Jetson, Coral TPU, Intel NUC)
- Execution providers (TensorRT, OpenVINO, XNNPACK)
- Analysis tools (power modeling, thermal prediction)

Contributions are accepted through GitHub issues and pull requests.

7. Illustrative Examples

7.1. Example 1: Latency–Accuracy Trade-off

Figure 3 shows mean, $p50$, and $p90$ latencies for all model–dataset combinations at batch=1, threads=4. MobileNetV3-Small achieves ∼2 ms latency but lower accuracy on CIFAR-100 (45.8%) compared to ResNet-18 (60.8%, 10.5 ms). This quantifies the latency–accuracy trade-off for SLO-aware model selection.

7.2. Example 2: Oracle–Device Gap

Table 5 shows oracle (Colab CPU) vs. Pi throughput for ResNet-18 on CIFAR-100. At batch=1, threads=4, the Pi achieves 93.7 img/s vs. oracle 284.2 img/s, yielding Speedup = 0.33 and ThermalLoss = 0.67. This quantifies the performance penalty from thermal constraints and limited cache.

7.3. Example 3: Batch Scaling

Figure 4 demonstrates throughput and latency vs. batch size for MobileNetV3-Small on CIFAR-100. Throughput increases from 450 img/s (batch=1) to 725 img/s (batch=4), following the amortization model $S(b) = S_\infty + K/b$. This guides deployment: use batch=1 for latency-critical applications, batch=4 for throughput maximization.

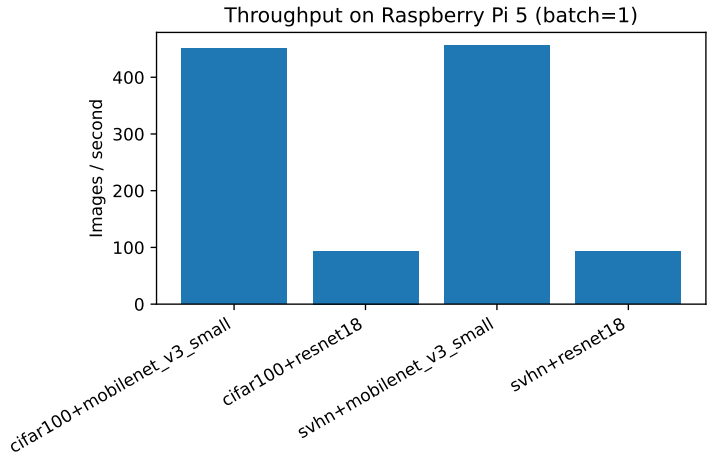


Figure 3: Latency comparison on Raspberry Pi 5 (batch=1, threads=4).

Table 5: Oracle–Pi gap for ResNet-18, CIFAR-100, batch=1, threads=4

Context	**Throughput (img/s)**	**Latency (ms)**
Oracle (Colab CPU)	284.2	3.52
Pi 5 (bare metal)	93.7	10.54
Speedup	0.33	–
ThermalLoss	0.67	–

8. Impact and Related Work

8.1. Impact on Research Community

This software enables reproducible edge AI research by:

- Providing a reference implementation for cross-device benchmarking
- Establishing a manifest-based contract for preprocessing and execution
- Quantifying oracle–device gaps often ignored in prior work
- Releasing validated artifacts for independent verification

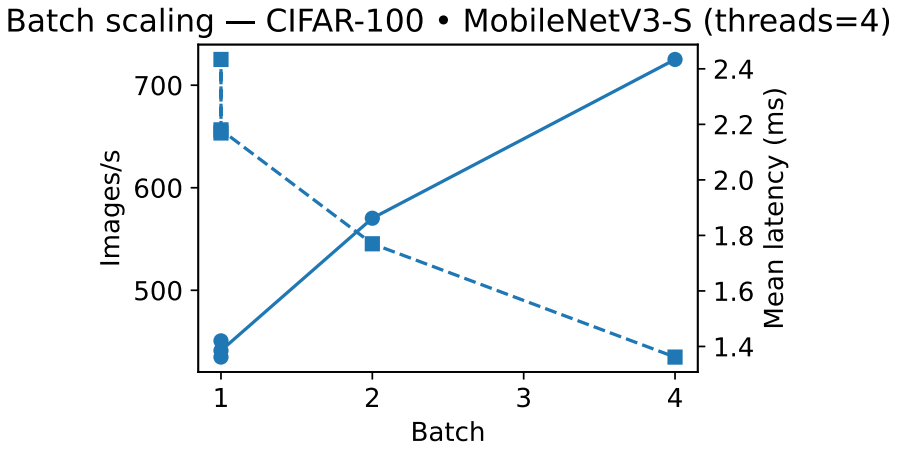


Figure 4: Batch scaling for MobileNetV3-Small on CIFAR-100.

8.2. Comparison with Existing Tools

Table 6 compares our software with related benchmarking frameworks.

Table 6: Comparison with existing edge AI benchmarking tools

Tool	**Artifacts**	**Oracle**	**Manifest**
MLPerf Edge [11]	Partial	No	No
MLPerf Tiny [12]	No	No	No
TVM AutoTVM [13]	Code only	No	No
This work	Full	Yes	Yes

Key differentiators:

- **MLPerf:** Defines closed workloads but does not release trained models or preprocessing code
- **TVM:** Focuses on kernel optimization, not end-to-end deployment
- **Our software:** Provides complete training→deployment→analysis pipeline with validated artifacts

8.3. Related Software

- **ONNX Model Zoo** [5]: Pre-trained models without deployment scripts or benchmarking
- **TensorFlow Model Garden**: Training code without edge deployment or gap analysis
- **PyTorch Hub**: Model definitions without ONNX export or manifest system

Our contribution is the *integration* of training, export, validation, deployment, and analysis into a single reproducible workflow.

9. Availability and Licensing

9.1. Software Availability

- **Repository:** `https://github.com/anonymous/edge-onnx-pipeline` (anonymized for review)
- **Zenodo DOI:** To be assigned upon acceptance

9.2. Licensing

- **Code:** MIT License (permissive, commercial use allowed)
- **Documentation:** Creative Commons CC-BY-4.0
- **Trained models:** Released under same license as training data (CIFAR-100 and SVHN are public domain)

9.3. Maintenance and Support

The repository will remain publicly available with the following support:

- **Repository:** Hosted on GitHub under MIT License
- **Documentation:** Installation guide and usage examples in the README; architecture and reproducibility notes in `docs/`

- **Contributions:** Community contributions welcome via pull requests
- **Issues:** Bug reports and feature requests tracked via GitHub Issues

10. Reproducibility Checklist

10.1. Complete Execution Protocol

10.1.1. Prerequisites

- Raspberry Pi 5 (2 GB or 8 GB) with Raspberry Pi OS Bookworm 64-bit
- 32 GB microSD card (Class 10 or better)
- 27 W USB-C power supply
- Internet connection for dependency installation
- Optional: INA219 sensor for power measurement

10.1.2. Exact Commands

Listing 4: Complete reproduction script

```
# 1. Clone repository
git clone https://github.com/anonymous/edge-onnx-pipeline.
    git
cd edge-onnx-pipeline

# 2. Download pre-trained artifacts (or train from scratch)
wget https://zenodo.org/record/XXXXX/bundle.zip
unzip bundle.zip

# 3. Validate manifest
python3 -m venv venv
source venv/bin/activate
pip install jsonschema
python3 validate_manifest.py MANIFEST.json
```

```
# 4. Run full benchmark
chmod +x validate_and_run.sh
./validate_and_run.sh

# 5. Generate analysis and figures
python3 analyze_results.py --input results/ --output figures/
```

10.1.3. Expected Runtime

Table 7: Expected runtime for each phase

Phase	**Duration**
Training (Colab, 4 models)	2–4 hours
ONNX export & validation	5–10 minutes
Pi setup (venv, deps)	10–15 minutes
Full inference grid (48 configs)	2.5–5 hours
Analysis & figure generation	2–5 minutes
Total (end-to-end)	**5–9 hours**

10.1.4. Expected Outputs

After successful execution, the following files will be generated:

- `results/pi_results_normalized.csv` (30 rows, 16 columns)
- `results/gap_analysis_with_speedup.csv` (21 rows, 16 columns)
- `figures/latency_bars.pdf`
- `figures/latency_cdf.pdf`
- `figures/throughput_bars.pdf`
- `figures/confusion_matrix_*.png` (4 files)

- `logs/thermal_trace_*.csv` (timestamped)
- `logs/power_trace_*.csv` (if INA219 present)

10.1.5. Result Storage

All outputs are stored in:

- `results/`: CSV files with metrics
- `figures/`: PDF/PNG plots
- `logs/`: Raw telemetry traces
- `checkpoints/`: Trained model weights (if training from scratch)

10.2. Verification Procedure

To verify correctness:

1. **Manifest validation:** Ensure no schema errors
2. **Latency sanity check:** Verify 2 ms $<$ MobileNetV3 latency $<$ 3 ms
3. **Throughput consistency:** Check throughput $\times$ latency ≈ 1000
4. **Accuracy range:** Confirm CIFAR-100 accuracy $\in [40, 65]\%$, SVHN $\in [90, 97]\%$
5. **Figure generation:** Verify all PDFs render without errors

10.3. Deterministic Guarantees

The software guarantees deterministic results via:

- Fixed random seeds (1337 for training, 42 for evaluation)
- Pinned dependency versions in `requirements.txt`
- Deterministic CuDNN settings
- Manifest-encoded preprocessing
- Logged quantization fallback decisions

Run-to-run variation is expected to be $< 2\%$ for mean latency due to OS scheduling noise.

11. Broader Impact

11.1. Scientific Impact

This software enables reproducible edge AI research by providing a reference implementation that:

- **Standardizes measurement**: Manifest-based contracts ensure consistent preprocessing across studies
- **Quantifies device gaps**: Oracle–device comparison reveals thermal and virtualization effects often ignored
- **Facilitates fair comparison**: Researchers can benchmark new methods against validated baselines

11.2. Reuse Potential

The software enables researchers to:

- **Benchmark new models**: Drop in custom architectures while preserving measurement protocol
- **Test new edge hardware**: Extend deployment scripts to additional platforms (Jetson, Coral)
- **Evaluate runtime frameworks**: Compare ONNX Runtime against TFLite or TVM
- **Teach edge AI experimentation**: Use as teaching material for reproducible ML systems

12. Conclusion

We have presented an open-source software pipeline that automates edge AI experiments from training to deployment. The software addresses three critical gaps in edge AI research: lack of portable measurement contracts, opaque preprocessing, and missing oracle baselines.

The pipeline provides:

- **Complete automation:** Single-command execution from Colab training to Raspberry Pi deployment
- **Reproducibility tooling:** JSON schema validation, deterministic execution, and manifest-based configuration
- **Extensible architecture:** Modular design supporting new models, datasets, and hardware platforms
- **Open release:** All components available under MIT License with documentation

The software enables researchers to replicate edge AI experiments exactly, benchmark new methods against validated baselines, and extend the pipeline to new hardware platforms without modifying measurement code.

Acknowledgments

This work was conducted on personal hardware and Google Colab free tier. We thank the ONNX Runtime and PyTorch communities for excellent documentation and support.

References

[1] A. Howard, M. Sandler, G. Chu, et al., Searching for MobileNetV3, in: Proc. ICCV, 2019.

[2] K. He, X. Zhang, S. Ren, J. Sun, Deep Residual Learning for Image Recognition, in: Proc. CVPR, 2016.

[3] A. Krizhevsky, Learning Multiple Layers of Features from Tiny Images, Technical Report, 2009.

[4] Y. Netzer, T. Wang, A. Coates, et al., Reading Digits in Natural Images with Unsupervised Feature Learning, NIPS Workshop on Deep Learning, 2011.

[5] ONNX Runtime documentation, https://onnxruntime.ai/docs/, accessed 2025.

[6] Y. Kang, J. Hauswald, C. Gao, et al., Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge, in: Proc. ASPLOS, 2017.

[7] Y. Matsubara, D. Callegaro, M. Levorato, F. Restuccia, Split Computing and Early Exiting for Deep Learning: Survey and Research Challenges, ACM Comput. Surveys, 2022.

[8] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed Deep Neural Networks over the Cloud, the Edge and End Devices, in: Proc. ICDCS, 2017.

[9] B. Jacob, S. Kligys, B. Chen, et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, in: Proc. CVPR, 2018.

[10] T. Benoit-Cattin, D. Velasco-Montero, J. Fernández-Berni, Impact of Thermal Throttling on Long-Term Visual Inference in a CPU-based Edge Device, arXiv:2010.06291, 2020.

[11] MLCommons, MLPerf Inference: Edge, https://mlcommons.org/benchmarks/inference-edge/, accessed 2025.

[12] C. Banbury, V. Jain, et al., MLPerf Tiny Benchmark, NeurIPS Datasets and Benchmarks, 2021.

[13] T. Chen, T. Moreau, Z. Jiang, et al., TVM: An Automated End-to-End Optimizing Compiler for Deep Learning, in: Proc. OSDI, 2018.

[14] TensorFlow Model Optimization Toolkit: Post-Training Quantization, https://www.tensorflow.org/model_optimization/guide/quantization/post_training, accessed 2025.

[15] OpenVINO Toolkit Documentation, https://docs.openvino.ai/, accessed 2025.